



### Abstract

- What it does:** An AI-powered web platform that matches job seekers to roles from their resume and lets them practise mock interviews. spaCy parses resumes, sentence-transformers rank matches, a Groq/Gemini LLM writes role-specific questions and faster-whisper transcribes spoken answers.
- Outcome:** One free-tier, open-source pipeline from resume to scored feedback on content and delivery, with progress stored in MongoDB.

### Problem Statement & Objectives

- Problem:** 75%+ of resumes are filtered out by ATS or recruiters before a human reads them; candidates then prepare with generic questions and get no feedback on pace or delivery.
- Obj. 1:** Parse resumes accurately and rank jobs by semantic fit, not keywords.
- Obj. 2:** Generate role-specific questions from the matched job and the candidate's skill gaps.
- Obj. 3:** Score spoken answers on content and delivery, and track progress over time.

### Existing vs Proposed System

| Feature      | Existing Systems                               | Proposed System                                 |
|--------------|------------------------------------------------|-------------------------------------------------|
| Job matching | ATS keyword filters; research stops at ranking | Semantic matching with matched & missing skills |
| Questions    | Generic question banks                         | LLM questions tied to the job and resume gaps   |
| Feedback     | Peer-only (Pramp) or delivery-only (Yoodli)    | Content + delivery scores: pace, filler words   |
| Cost         | Credit-metered / paid coaching                 | Free-tier, open-source stack                    |
| Gap          | No tool links resume → job → practice          | One integrated pipeline with stored progress    |

### Feasibility Study

- Technical:** React, Node/Express, Flask and Python NLP libraries are all open-source and well documented.
- Economic:** Free tiers (MongoDB Atlas, Groq/Gemini); runs on an 8 GB RAM laptop.
- Operational:** Browser-only, no plugins; simple dashboard for candidates.
- Schedule:** Built and demonstrated within Trimester 4, 2026-27.

### Tools & Technologies

- Front-End:** React.js; getUserMedia + MediaRecorder for mic/camera capture
- Back-End:** Node.js + Express REST API; Python Flask AI microservice
- AI / NLP:** spaCy, sentence-transformers, faster-whisper
- Database:** MongoDB Atlas (sessions, transcripts, scores)
- Tools:** Git & GitHub, Postman, Vite, MongoDB Atlas data viewer

### System Architecture

#### Presentation Layer – Frontend (React)



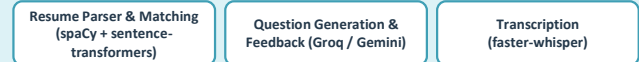
▼ REST calls, auth via Node

#### Application Layer – Backend (Node.js + Express)

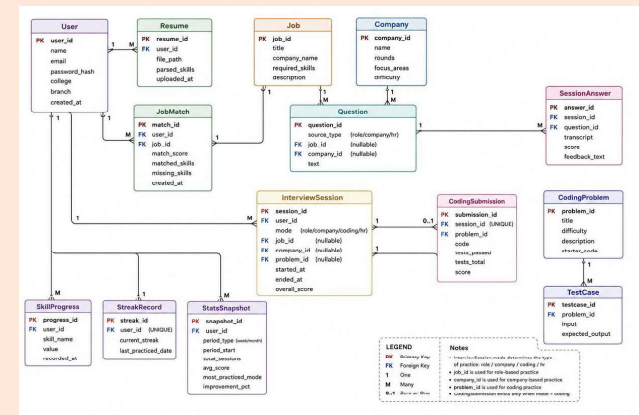


▼ REST: orchestrates AI service calls

#### AI Service Layer – Python / Flask microservice



### ER Diagram & Database Design



14 entities linked by IDs (one User → many Resumes, Matches, Sessions); indexed on candidate and session for fast dashboard lookups.

### Implementation Screenshots

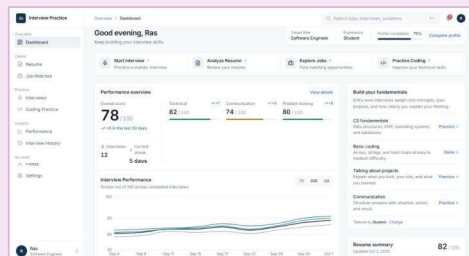


Fig. 1 Candidate dashboard: readiness score, skills, trend

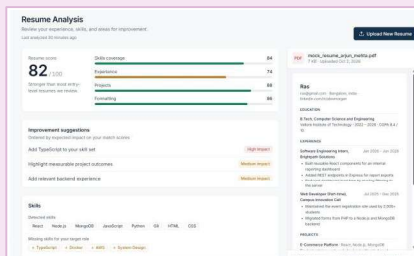


Fig. 2 Resume analysis: score, skills, improvement tips

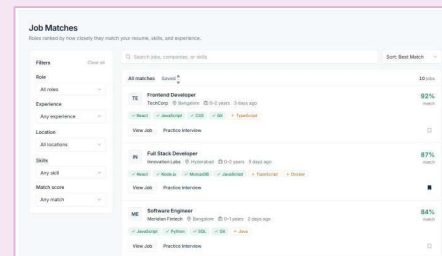


Fig. 3 Job matches ranked by semantic fit, matched / missing skills

### Results & Testing

- Postman tests on all Node and Flask REST endpoints.
- End-to-end run: upload → match → question → answer → feedback → dashboard.
- 10 test cases: upload validation, matching, transcription, rate limits, browsers.
- Demo: resume score 82/100, top match 92%.

### Conclusion

- Delivered a personalised pipeline from resume to ranked roles, tailored questions, recorded answers and feedback.
- Meets all objectives: semantic matching, role-specific questions, content + delivery scoring, progress tracking on a free-tier stack.

### Future Scope

- Body-language analysis (MediaPipe) and voice-modulation scoring (librosa).
- Recruiter dashboard and live job-board integration (LinkedIn / Indeed).
- Multi-language interviews and peer mock scheduling.